

# **INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)**

## **Lab 3 - SYN Flood Pattern Investigation Using TShark**

|                      |                                                      |
|----------------------|------------------------------------------------------|
| <b>Lab Title:</b>    | <b>SYN Flood Pattern Investigation</b>               |
| <b>Student Name:</b> | <b>Yusuf Oluwaseun Dada</b>                          |
| <b>Student ID:</b>   | <b>2025/FWSD/11339</b>                               |
| <b>Course Code:</b>  | <b>SBT-DF203</b>                                     |
| <b>Course Name:</b>  | <b>Basic Networking Skills for Digital Forensics</b> |

**Instructor: Mr. Aminu Idris, AMCPN**

**Date: 11/09/2026**

## 1. Lab Folder Setup and Tools

```
(kali@kali)-[~/Desktop]
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab3
pwd
/home/kali/SBT-DF203-Lab3
find . -maxdepth 1 -type d -print
./exported
./evidence
./working
./screenshots
./scripts
./reports

CGroup: /system.slice/apache2.service
├─ 946 /usr/sbin/apache2 -k start -DFOREGROUND
├─ 22347 /usr/sbin/apache2 -k start -DFOREGROUND
├─ 22348 /usr/sbin/apache2 -k start -DFOREGROUND
├─ 22349 /usr/sbin/apache2 -k start -DFOREGROUND
├─ 22350 /usr/sbin/apache2 -k start -DFOREGROUND
└─ 22351 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 11 13:46:31 kali systemd[1]: Starting apache2.service - The Apache HTTP Server
Sep 11 13:46:35 kali apachectl[946]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead; this will likely move into the future.
Sep 11 13:46:36 kali systemd[1]: Started apache2.service - The Apache HTTP Server
Sep 11 14:28:46 kali systemd[1]: Reloading apache2.service - The Apache HTTP Server
Sep 11 14:28:50 kali apachectl[22282]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.0.1 instead; this will likely move into the future.
Sep 11 14:28:50 kali systemd[1]: Reloaded apache2.service - The Apache HTTP Server
Hint: Some lines were ellipsized, use -l to show in full.
LISTEN 0      511      *:::80      *:*        users:((("apache2",pid=22351,fd=4),("apache2",pid=22350,fd=4),("apache2",pid=22349,fd=4),("apache2",pid=22348,fd=4),("apache2",pid=22347,fd=4),("apache2",pid=946,fd=4)))
HTTP/1.1 200 OK
Date: Fri, 11 Sep 2026 18:29:39 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Thu, 10 Sep 2026 02:32:56 GMT
ETag: "2a03-65b17c91c8fa4"
Accept-Ranges: bytes
Content-Length: 10755
Vary: Accept-Encoding
Content-Type: text/html
```

Copy the authorised mySYNFloodCapture.pcap into the evidence folder. Use the real path on your Kali VM.

```
cp /kali/home/mySYNFloodCapture.pcap ~/SBT-DF203-Lab3/evidence/
cd ~/SBT-DF203-Lab3
ls -lh evidence/
```

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -lh evidence/
total 256K
-rw-r--r-- 1 kali kali 255K Sep 11 11:43 mySYNFloodCapture.pcap

(kali@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 evidence/mySYNFloodCapture.pcap
```

## 2. Chain-of-Custody Worksheet

| Field                       | Your Entry                                                       |
|-----------------------------|------------------------------------------------------------------|
| Case/lab identifier         | SBT-DF203-Lab3                                                   |
| Trainee name                | Yusuf Oluwaseun Dada                                             |
| Date and time started       | 09/09/2026 4:30pm                                                |
| Evidence file name(s)       | mySYNFloodCapture.pcap                                           |
| Source or generation method | Downloaded reference PCAP / self-generated via tshark+curl+Scapy |
| Original SHA-256            | 14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5 |
| Working-copy SHA-256        | 9dcc96ea6430a7f939ff98d39602d6f2b5a269bbb5cb6743819964e703a99fa0 |
| Analysis workstation/VM     | eKali Linux VM, host-only                                        |
| Notes on any changes        |                                                                  |

### 3. Part A - Normal HTTP Handshake Baseline

sudo tshark -i lo -f 'tcp port 80' -a duration:15 -w /normal\_http.pcapng & sleep 2  
curl --no-keepalive http://127.0.0.1/ >/dev/null wait

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ cd ~/SBT-DF203-Lab3
sudo tshark -i lo -f "tcp port 80" -a duration:15 -w /normal_http.pcapng &
sleep 2
curl --no-keepalive http://127.0.0.1/ >/dev/null
wait
[2] 180171
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  10755  100  10755    0     0  128.5k      0   10      0
[2]  done      sudo tshark -i lo -f "tcp port 80" -a duration:15 -w /normal_http.pcapng

(kali@kali)-[~/SBT-DF203-Lab3]
$ ls -la /normal_http.pcapng
-rw-r--r-- 1 root root 12452 Sep 11 20:04 /normal_http.pcapng
```

TShark captures loopback traffic involving TCP/80 for 15 seconds. curl generates one normal HTTP request. The output capture provides the baseline.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ cd ~/SBT-DF203-Lab3
sudo mv /normal_http.pcapng evidence/normal_http.pcapng
ls -la evidence/normal_http.pcapng
[sudo] password for kali:
-rw-r--r-- 1 root root 12452 Sep 11 20:04 evidence/normal_http.pcapng
```

Moving it from root path to the evidence directory

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ sha256sum evidence/normal_http.pcapng | tee reports/normal_http_sha256.txt
289f1e27f558181a739b195286c745339cd0078e6b418787066b4a7637aa8f35 evidence/normal_http.pcapng
```

sha256sum evidence/normal\_http.pcapng | tee reports/normal\_http\_sha256.txt

289f1e27f558181a739b195286c745339cd0078e6b418787066b4a7637aa8f35

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r evidence/normal_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields -e frame.number -e frame.time -e ip.src -e tcp.src
-e ip.dst -e tcp.dstport -e tcp.flags | tee reports/normal_handshake_flags.tsv
1  2026-09-11T20:04:28.968645138-0400  127.0.0.1  40104  127.0.0.1  80  0x0002
2  2026-09-11T20:04:28.968675959-0400  127.0.0.1  80  127.0.0.1  40104  0x0012
8  2026-09-11T20:04:29.038328745-0400  127.0.0.1  40104  127.0.0.1  80  0x0011
9  2026-09-11T20:04:29.038432883-0400  127.0.0.1  80  127.0.0.1  40104  0x0011
```

tshark -r evidence/normal\_http.pcapng -Y 'tcp.flags.syn==1 || tcp.flags.fin==1' -T fields \

-e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \

| tee reports/normal\_handshake\_flags.tsv

The display filter selects SYN packets. The -e options extract frame number, time, source/destination IPs and ports, and TCP flags.

## 4. Part B - Conduct the Bounded Loopback Simulation

This creates and runs the four-packet Scapy script, exactly as authorized, not to be modified to send more packets, loop continuously, or target another host.

```
GNU nano 9.2 syn_probe_lab.py *
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(),
dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
```

The script crafts four IP/TCP packets, each with the SYN flag set and a random source port (RandShort()), and sends them to 127.0.0.1:80. Setting COUNT = 4 hard-bounds the simulation, this is the safety control that keeps the exercise a controlled test rather than a real flood.

On first terminal

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ sudo tshark -i lo -f 'tcp port 80' -c 20 -w /bounded_syn_activity.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
12 ^C
```

Terminal 1 starts TShark listening on loopback for port-80 traffic, capped at -c 20 packets (a safety ceiling, not a target)

On second terminal

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ sudo python3 scripts/syn_probe_lab.py
Sent 4 authorized training SYN packets to 127.0.0.1:80
```

Terminal 2 then runs the bounded Scapy script, which sends the four SYN packets. TShark records the SYNs and Apache's responses to them.

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ sudo mv /bounded_syn_activity.pcapng evidence/bounded_syn_activity.pcapng
ls -la evidence/bounded_syn_activity.pcapng
-rw-r--r-- 1 root root 1428 Sep 11 21:20 evidence/bounded_syn_activity.pcapng
```



## 5. Copy the Evidence and Hash It

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
9dcc96ea6430a7f939ff98d39602d6f2b5a269bbb5cb6743819964e703a99fa0  evidence/
e/bounded_syn_activity.pcapng
9dcc96ea6430a7f939ff98d39602d6f2b5a269bbb5cb6743819964e703a99fa0  working
/bounded_syn_activity_working.pcapng
```

```
PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee
reports/bounded_capture_hashes.txt
```

This is the core evidence-integrity step. The original capture stays untouched in evidence/; cp --preserve=timestamps makes a working copy (preserving the original modification time) in working/, which is what i actually run analysis commands against. sha256sum then hashes both the original and the copy, and the two hashes match, which is bit-for-bit identical to the original evidence.

## 6. Part C - Identify SYN Indicators with TShark

Four extraction commands isolate each stage of the handshake attempt from the bounded capture.

### 6.1 Initial SYNs only

```
(kali㉿kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst
-e tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv
1      1789175928.482628580      127.0.0.1      42385      127.0.0.1      8
0      0
4      1789175928.514418259      127.0.0.1      24602      127.0.0.1      8
0      0
7      1789175928.517902083      127.0.0.1      19690      127.0.0.1      8
0      0
10     1789175928.520598150      127.0.0.1      60609      127.0.0.1      8
0      0
```

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -
e tcp.seq \
  | tee reports/initial_syns.tsv
```

The filter tcp.flags.syn==1 && tcp.flags.ack==0 matches only the first packet of a handshake, a bare SYN with no ACK flag set, which is what a client normally sends to

open a connection. Extracting tcp.seq (the initial sequence number) alongside timing and port data and matches each SYN to its corresponding SYN-ACK reply.

## 6.2 SYN-ACK responses

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -
  e tcp.dstport -e tcp.ack \
  | tee reports/syn_ack_responses.tsv
2      1789175928.482769844      127.0.0.1      80      127.0.0.1      42
385    1
5      1789175928.514526289      127.0.0.1      80      127.0.0.1      24
602    1
8      1789175928.517981829      127.0.0.1      80      127.0.0.1      19
690    1
11     1789175928.520657179      127.0.0.1      80      127.0.0.1      60
609    1
```

```
tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -
  e tcp.ack \
  | tee reports/syn_ack_responses.tsv
```

This filter matches packets with both SYN and ACK flags set, the server's reply to a SYN, step two of the handshake. Comparing the count of these against the initial-SYN count

## 6.3 Resets and final-ACK candidates

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.l
en==0)' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -
  e tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv
2      1789175928.482769844      127.0.0.1      80      127.0.0.1      42
385    0x0012
3      1789175928.510343124      127.0.0.1      42385      127.0.0.1      80
0x0004
5      1789175928.514526289      127.0.0.1      80      127.0.0.1      24
602    0x0012
6      1789175928.514553302      127.0.0.1      24602      127.0.0.1      80
0x0004
8      1789175928.517981829      127.0.0.1      80      127.0.0.1      19
690    0x0012
9      1789175928.518008225      127.0.0.1      19690      127.0.0.1      80
0x0004
11     1789175928.520657179      127.0.0.1      80      127.0.0.1      60
609    0x0012
12     1789175928.520680696      127.0.0.1      60609      127.0.0.1      80
0x0004
```

This checks for two things at once: RST packets (a connection being forcibly torn down) and bare ACKs with zero payload length (the pure handshake-completing ACK rather than a data-carrying packet). Together these tells whether any of the four

SYNs actually resulted in a completed, normal connection close, or whether the server (or OS) eventually reset the half-open attempts.

## 7. Part D - Quantify and Correlate the Activity

### 7.1 Counts by source/destination pair

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt
  4 127.0.0.1      127.0.0.1      80
```

Pipes the raw SYN list through `sort | uniq -c`, which counts how many identical source>>destination:port lines occur, like how many SYNs each client IP sent to the target. `sort -nr` then ranks the busiest source first. This is the number an analyst would use to flag a single host sending an abnormally high SYN rate.

### 7.2 Unique client source ports

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt
19690
24602
42385
60609
```

It extracts just the source ports of the SYN packets and reduces them to a unique, sorted list. A flood or scan from one machine typically uses a fresh random source port per connection attempt (as this lab's Scapy script does with `RandShort()`), so the number of unique ports roughly equals the number of connection attempts from that host, which is a useful indicator on its own.



## 7.3 Expert information and TCP analysis flags

```
(kali@kali)~[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
  -T fields -e frame.number -e frame.time -e _ws.col.Info \
  | tee reports/tcp_analysis_events.tsv

Warns (4)
=====
Frequency      Group          Protocol  Summary
         4      Sequence          TCP      Connection reset (RST)

Notes (4)
=====
Frequency      Group          Protocol  Summary
         4      Protocol          TCP      The SYN packet does not contain a SACK PERM option

Chats (8)
=====
Frequency      Group          Protocol  Summary
         4      Sequence          TCP      Connection establish request (SYN): server port 80
         4      Sequence          TCP      Connection establish acknowledgment (SYN+ACK): server port 80
```

Note:

-z expert runs Wireshark's built-in expert-analysis engine in quiet mode (-q suppresses the normal packet list), summarizing anomalies such as retransmissions or out-of-order segments it noticed automatically. The second command filters specifically for TShark's own analysis flags, retransmissions, lost segments, duplicate ACKs, which are automatically generated diagnostic markers, not raw flags in the packet itself, and are strong corroborating evidence of connection trouble.

## 8. Part E - Compare Normal and Suspicious Sessions

| Indicator            | Normal HTTP Session                           | Bounded SYN Activity                                                    | Forensic Meaning                                    |
|----------------------|-----------------------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------|
| Initial SYN count    | 1 (from normal_handshake_flags.tsv)           | 4 (from initial_syns.tsv)                                               | Number of initial connection attempts               |
| SYN-ACK count        | It shows complete the TCP three way handshake | It shows repeated SYN packets and may create many half open connections | Server responses                                    |
| Completed handshakes | 1                                             | 4                                                                       | Connections where the final ACK is observed/matched |

| Indicator                  | Normal HTTP Session    | Bounded SYN Activity                         | Forensic Meaning                 |
|----------------------------|------------------------|----------------------------------------------|----------------------------------|
| Unique client source ports | One client source      | Four unique client source port were observed | Distinct client TCP source ports |
| HTTP request present?      | Yes, from curl request |                                              | Shows application-layer traffic  |
| RST packets                |                        |                                              | Connection resets                |
| Observed duration          |                        |                                              | Time covered by the capture      |

## 9. Executive Summary

This practical investigated TCP SYN activity using TShark in an authorised isolated training environment. A normal HTTP session was first captured to establish a TCP handshake baseline. A bounded four-packet SYN simulation was then performed against the local Apache service on 127.0.0.1:80. The captures were examined for SYN, SYN-ACK, ACK and RST behaviour, source ports, packet counts and timing. SHA-256 hashes were calculated to support evidence integrity. The results were compared to identify characteristics associated with incomplete TCP connection attempts. The bounded simulation was not treated as proof of a real SYN flood because it lacked the scale, persistence and service impact required to establish a denial-of-service event.

## 10. Detection and Mitigation Recommendations

### Detection:

- ✓ Alert on an elevated SYN-to-completed-handshake ratio per source and destination.
- ✓ Monitor SYN backlog exhaustion, retransmissions, connection-state table size, and service latency.

### Recommendations:

- ✓ To use SYN cookies, sensible backlog/timeout tuning, upstream rate limiting, and DDoS protection where appropriate.
- ✓ To retain full packet capture or flow telemetry during incidents, and keep system clocks synchronized for accurate timelines.
- ✓ Correlate packet-level evidence with web server, firewall, load balancer, and operating-system logs.